Nama :Veva Br Tarigan

NIM :253303621280

Kelas :3 Pagi B

Jurusan :Sistem Informasi

Mata Kuliah :Dr.Evta Indra,S.Kom.,M.Kom.

JAVA VISUAL AP6 D

1. ntuk kode Character a = new Character(...); Character b = a; b.setLevel(8);, jelaskan apa yang dicetak a.getLevel() dan a == b beserta alasannya memakai istilah *reference*. Bagaimana membuat b benar-benar independen?

   Jawaban:

   1. Pengertian Reference pada Java

   Dalam bahasa pemrograman Java, *reference* adalah nilai yang digunakan oleh variabel untuk merujuk pada suatu objek. Ketika sebuah objek dibuat menggunakan kata kunci new, Java membuat objek tersebut di memori. Variabel yang digunakan untuk menyimpan hasil pembuatan objek akan menyimpan reference menuju objek tersebut.

   Reference penting untuk dipahami karena beberapa variabel dapat menunjuk ke objek yang sama. Jika salah satu variabel mengubah keadaan objek tersebut, perubahan itu juga dapat dilihat melalui variabel lain yang menunjuk ke objek yang sama.

   Pada class Character, objek memiliki atribut seperti nama, tipe karakter, level, hit points, attack power, dan mana. Atribut tersebut dapat diakses atau diubah melalui method yang tersedia, misalnya getLevel() untuk mengambil level dan setLevel() untuk mengubah level.

   2. Analisis Kode Character a dan Character b

   Perhatikan kode Java berikut:

   ```
   Character a = new Character("Aragorn", CharacterType.WARRIOR, 5);
   Character b = a;

   b.setLevel(8);

   System.out.println(a.getLevel());
   System.out.println(a == b);
   ```

   Pada baris pertama, program membuat objek Character baru dengan nama Aragorn, tipe WARRIOR, dan level awal 5. Reference menuju objek tersebut disimpan dalam variabel a.

   Pada baris kedua, Character b = a; menyalin reference yang dimiliki oleh a ke variabel b. Baris ini tidak membuat objek baru. Akibatnya, variabel a dan b menunjuk pada objek Character yang sama.

Pada baris ketiga, b.setLevel(8); mengubah level objek yang ditunjuk oleh b menjadi 8. Karena a juga menunjuk pada objek yang sama, perubahan level tersebut dapat dilihat melalui variabel a.

3. Hasil Output dan Penjelasannya

Output dari program tersebut adalah:

8

true

a. Hasil a.getLevel()

Nilai yang dicetak adalah 8. Hal ini terjadi karena variabel a dan b memiliki reference yang menunjuk ke objek yang sama. Ketika level objek diubah menjadi 8 melalui b.setLevel(8), level objek yang sama juga terbaca sebagai 8 melalui a.getLevel().

b. Hasil a == b

Nilai yang dicetak adalah true. Pada objek Java, operator == membandingkan apakah kedua variabel merujuk pada objek yang sama. Karena a dan b menunjuk pada satu objek yang sama, hasil perbandingannya adalah true.

Dengan demikian, kode Character b = a; hanya menyalin reference, bukan menyalin seluruh objek menjadi dua objek yang terpisah.

4. Cara Membuat Objek b Benar-Benar Independen

Agar objek b independen dari objek a, program harus membuat objek baru menggunakan kata kunci new. Contohnya:

```
Character a = new Character("Aragorn", CharacterType.WARRIOR, 5);
Character b = new Character("Aragorn", CharacterType.WARRIOR, 5);

b.setLevel(8);

System.out.println("Level a: " + a.getLevel());
System.out.println("Level b: " + b.getLevel());
System.out.println("Reference sama: " + (a == b));
```

Output program:

Level a: 5

Level b: 8

Reference sama: false

Pada kode tersebut, a dan b dibuat melalui dua pemanggilan new Character(...). Oleh karena itu, keduanya merupakan dua objek yang berbeda. Perubahan level pada objek b tidak mengubah level objek a.

Walaupun kedua objek memiliki nama dan tipe karakter yang sama, keduanya tetap merupakan objek yang berbeda di memori. Hal ini dibuktikan oleh hasil a == b yang bernilai false.

Cara ini membuat kedua objek independen dalam hal perubahan atributnya, selama tidak ada atribut yang sengaja berbagi objek mutable yang sama.

2. Beri analogi sendiri tentang perbedaan reference dan primitive value (boleh berbeda dari analogi di materi)
   Jawaban:
   Analogi Perbedaan Reference dan Primitive Value
   Menurut saya, perbedaan reference dan primitive value dapat dianalogikan seperti alamat rumah dan nomor rumah yang ditulis di kertas.
   1. Primitive Value
   Primitive value seperti int, double, dan boolean menyimpan nilai secara langsung. Analogi sederhananya adalah sebuah kertas yang berisi angka uang, misalnya Rp10.000. Jika angka pada kertas pertama diubah menjadi Rp20.000, kertas kedua yang berisi Rp10.000 tidak ikut berubah karena keduanya menyimpan nilai masing-masing.

   Contoh kode:
   ```
   int a = 10;
   int b = a;

   b = 20;

   System.out.println(a); // 10
   System.out.println(b); // 20
   ```
   2. Reference
   Reference dapat dianalogikan seperti alamat rumah yang ditulis pada dua kertas berbeda. Kedua kertas tersebut menunjukkan rumah yang sama. Jika isi rumah diubah, perubahan itu dapat dilihat oleh orang yang menggunakan alamat dari kedua kertas tersebut.
   Contoh kode:
   ```
   Character a = new Character("Aragorn", CharacterType.WARRIOR, 5);
   Character b = a;

   b.setLevel(8);

   System.out.println(a.getLevel()); // 8
   System.out.println(a == b);       // true
   ```
   Pada kode tersebut, a dan b menunjuk ke objek Character yang sama. Ketika level diubah melalui b, perubahan tersebut juga terlihat melalui a.

3. Mengapa `totalCharacterCreated` bersifat `static`? Apa yang terjadi jika tidak static?

   Jawaban:

   Atribut totalCharacterCreated bersifat static karena digunakan untuk menghitung jumlah objek Character yang telah dibuat secara bersama-sama oleh seluruh objek dalam class

Character. Dengan menggunakan static, atribut tersebut dimiliki oleh class, bukan oleh masing-masing objek.

Contoh kode:

```
public class Character {

   private static int totalCharacterCreated = 0;



   public Character() {

      totalCharacterCreated++;

   }



   public static int getTotalCharacterCreated() {

      return totalCharacterCreated;

   }

}
```

Pada contoh tersebut, setiap kali constructor dipanggil untuk membuat objek baru, nilai totalCharacterCreated bertambah satu.

Apa yang terjadi jika tidak menggunakan static?

Jika atribut tersebut tidak menggunakan static, setiap objek akan memiliki salinan variabelnya sendiri. Akibatnya, penghitung tidak dapat digunakan sebagai satu penghitung bersama untuk mengetahui jumlah objek yang telah dibuat oleh seluruh class.

4. Mengapa equals() bawaan class Object tidak cocok untuk cek duplikat nama, dan apa solusinya?
   Jawaban:
   Method equals() bawaan class Object tidak cocok untuk mengecek duplikat nama karena secara default method tersebut membandingkan identitas objek, bukan nama atau isi objek. Contohnya, dua objek Character dapat memiliki nama yang sama, tetapi tetap dianggap berbeda oleh equals() bawaan karena keduanya merupakan objek yang berbeda.
   Solusinya adalah melakukan override method equals() pada class Character agar membandingkan atribut name.

Contoh kodenya:

```
@Override
public boolean equals(Object obj) {
    if (this == obj) {
        return true;
    }

    if (!(obj instanceof Character)) {
        return false;
    }

    Character other = (Character) obj;
    return this.name.equals(other.name);
}
```

Dengan cara ini, dua objek Character yang memiliki nama sama akan dianggap sama berdasarkan nama. Selanjutnya, method addCharacter() pada CharacterManager dapat menggunakan equals() untuk menolak karakter dengan nama duplikat.

5. Apa risiko jika `getAllCharacters()` mengembalikan ArrayList asli? Kaitkan dengan konsep reference di nomor 1

Jawaban:
Jika method getAllCharacters() mengembalikan ArrayList asli, pengguna dapat mengubah daftar karakter secara langsung dari luar class CharacterManager. Contohnya, jika pengguna menjalankan clear(), semua karakter dalam daftar asli dapat terhapus tanpa melalui method removeCharacter().
Hal ini berkaitan dengan konsep reference pada nomor 1. Ketika sebuah variabel menerima reference ke ArrayList asli, kedua variabel menunjuk ke objek daftar yang sama. Akibatnya, perubahan melalui satu variabel akan memengaruhi daftar yang digunakan oleh CharacterManager.
Solusinya adalah menggunakan defensive copy, yaitu mengembalikan salinan daftar dengan kode berikut:

```
public ArrayList<Character> getAllCharacters() {
    return new ArrayList<>(characters);
}
```

Dengan cara ini, perubahan pada struktur daftar salinan tidak akan mengubah daftar asli. Namun, objek Character di dalam kedua daftar masih bisa memiliki reference yang sama, sehingga perubahan atribut suatu karakter masih dapat memengaruhi objek yang sama.